Retrieval-Augmented Generation (RAG)

Introduction to RAG

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with language model generation.

A RAG system retrieves relevant information from an external knowledge source and gives that information to an LLM before generating an answer.

Why RAG is Used

• Answer questions using private or domain-specific documents

• Reduce dependence on information stored only in the model

• Build question-answering systems over PDFs, books, notes, and company documents

• Provide relevant context to an LLM

RAG Pipeline

A typical RAG pipeline follows these steps:

• Load documents from sources such as PDF, TXT, CSV, or web pages

• Split documents into smaller chunks

• Create embeddings for the chunks

• Store embeddings in a vector database

• Retrieve the most relevant chunks for a user query

• Send the query and retrieved context to an LLM

• Generate the final answer

Document Loaders

Document loaders read information from external files and convert it into a format that can be processed by a RAG pipeline.

For PDF files, PyPDFLoader can be used. For a directory containing multiple PDFs, DirectoryLoader can load the files and use PyPDFLoader for each PDF.

```
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader(
    "C:/Retrieval_augumented_generation/books",
    glob="*.pdf",
    loader_cls=PyPDFLoader
)

documents = loader.load()
```

Text Splitting

Large documents are usually divided into smaller chunks before creating embeddings. Chunking makes retrieval more focused and helps the system provide relevant context.

Embeddings

An embedding model converts text into numerical vectors. Texts with similar meanings can have vectors that are close together in vector space.

These vectors allow a vector store to perform semantic similarity search.

Vector Stores and Retrieval

A vector store saves embeddings and supports similarity search. A retriever uses a user's query to find the most relevant document chunks.

The retrieved chunks become context for the language model.

Generation

The LLM receives the user's question together with the retrieved context and generates a natural-language answer based on that information.

Simple RAG Flow

```
PDF / TXT / CSV
       ↓
Document Loader
       ↓
Text Splitter
       ↓
Embeddings
       ↓
Vector Store
       ↓
Retriever
       ↓
LLM
       ↓
Answer
```

RAG and Python

Python is frequently used to implement RAG systems. Frameworks such as LangChain can connect document loaders, text splitters, embedding models, vector stores, retrievers, and LLMs into an application.